Agentic Synthesis against Counterexample-Supplemented Sketches

Muness Castle Eric Rubeck

July 2026

Abstract

Coding agents make plausible repairs cheap. Teams still need a durable way to carry domain corrections into future runs. A correction should leave more than a passing test or a chat note: it should name the rejected repair, the rule that rejects it, and the code or prompt surface that must preserve it.

We use the term *counterexample-supplemented sketch* for the repository artifact and *agentic synthesis* for the loop that edits against it. A human writes a partial program-like sketch. Subject-matter experts correct concrete outputs. When a candidate failure contradicts or extends the sketch, an operator must explicitly approve it as a counterexample. Every accepted counterexample revises the sketch. A complete archive records that history, while a curated subset becomes executable regression checks. A coding agent edits deterministic code and prompt surfaces under the evolved sketch and known-code anchors.

The loop borrows from sketching and counterexample-guided inductive synthesis (CEGIS), but the execution surface is different. Classical sketching and CEGIS can make formal claims when the search space, specification, and checker are formal. Here, the synthesizer is an IDE-shaped coding agent whose edits span ordinary source code and LLM prompt surfaces. The claim stays bounded: when the gate passes, the current strategy satisfies the repository's encoded replay and comparison predicates for the current regression set.

The originating deployment ran inside a proprietary enterprise harness: a custom Cursor extension, `cursor://`-style commands, and an embedded web application where subject-matter experts loaded production examples, corrected outputs, and helped turn those corrections into input/output specifications. We evaluate the public method with CatSynth, a synthetic browser application and a captured coding-agent experiment. The experiment gives the Developer one active failure at a time, requires a sketch revision for each accepted counterexample, and gates every revision against a regression set. CatSynth is small, so its regression set contains its entire accepted archive. A closed-world run separately gives the same model a complete immutable specification; it reaches 20/20 visible and passes 21/21 withheld cases in four Developer calls and 611,519 model tokens through visible acceptance. In the open-world run, eight of fourteen externally supplied cases fail and are promoted. Tokens through visible acceptance are 891,880 for replay-all, 828,628 for evolved-sketch rebuild, and 1,021,822 for retained Sketch–CE. The controls inherit the promotion schedule and omit candidate classification and rule proposal, so these totals are not end-to-end price rankings. All three pass eight visible cases; their withheld-case results are 15/21, 19/21, and 18/21 respectively. These are inspectable runs, not general performance claims.

1 Introduction

Coding agents make repairs cheap before the repository has captured every domain rule the repair must obey. A developer can ask for a patch from a failing test, a state delta, a prompt instruction, or a reviewer note. The agent can satisfy that signal and still violate the rule that made an SME

reject the first repair. If the rule stays in chat or review memory, the next agent run can make the same plausible mistake.

The repair loop turns that rejected repair into a reviewed change to the governing model. An SME correction supplies the proposed output and the rule it exemplifies. If the case counters the current sketch, the system raises it to an operator. Only explicit approval gives the case specification authority. The Developer then revises the sketch, deterministic code, and prompt-mediated surface under known-code anchors. It receives one active failure. The gate receives the current regression set. No new counterexample is revealed until that gate is green.

Tests and prompts remain part of the loop. Tests catch shape errors, exceptions, bad deltas, and regressions. Prompts can state intent for the current run. An accepted counterexample has a different job: force a reviewed change to the sketch and preserve why that change happened. Regression cases are selected from the accepted archive to test later implementations.

The loop needs six artifacts:

1. a **sketch** $\mathcal{S}$: a partial program in prose and code-shaped structure;
2. an **accepted-counterexample archive** $\mathcal{A}$: the complete provenance of approved failures and corrected behavior;
3. a **regression set** $\mathcal{R} \subseteq \mathcal{A}$: selected executable cases that reject known wrong implementations;
4. **known-code anchors** $\mathcal{K}$: codebase-local shapes, APIs, conventions, and reference paths the agent must preserve;
5. a **dual-oracle implementation surface**: deterministic code for encodable holes and prompt-mediated completion for narrative holes;
6. a **gate** $\mathcal{G}$: replay plus semantic comparison over $\mathcal{R}$.

Method at a glance

Candidate failure counters the current sketch

⇓

Operator explicitly approves the counterexample and corrected behavior

⇓

Developer revises sketch $\mathcal{S}$, code, and prompt for that failure

⇓

Gate runs the active case and regression set $\mathcal{R}$

⇓

Green: curate $\mathcal{R}$ and reveal the next case; red: return one failed regression

The loop turns a tempting wrong repair into durable repository guidance.

The initial sketch records the human's current strategy: which repairs are permitted, prioritized, or forbidden. During synthesis, the Developer may reorganize or generalize that strategy while it edits code and prompts. Every accepted counterexample must leave a reviewed rule in the evolved sketch. The archive records all cases that earned specification authority. The regression set preserves a selected executable boundary. Operator-approved outputs remain authoritative; a model-drafted explanation cannot overrule them. The code is replaceable: a fresh implementation should be regenerable from the evolved sketch and known-code anchors, without replaying the archive as prompt context.

Claim. A repository-native workflow is inspectable when every accepted counterexample connects its operator decision, archive record, and reviewed sketch clause. Selected cases also connect to the replay check that verifies state repair and the semantic compare check that rejects a tempting wrong implementation. Passing the gate gives maintainers a bounded invariant over the current regression set, replay/checker code, and approved expected rows. It does not claim correctness outside those artifacts.

Boundary. The claim is bounded. Existing agentic workflows can make tests and prompts durable. Classical CEGIS already has counterexample discipline when its formal assumptions hold. Repository work is smaller and messier: the human strategy, the accepted counterexample, and the executable check often live in separate artifacts. The repair loop stores the links among them without treating the result as a solver-backed proof.

What the paper provides.

1. a repository-native artifact: the counterexample-supplemented sketch;
2. a process model that turns operator-approved corrections into an evolved sketch, a complete counterexample archive, and a curated regression set;
3. two hole-filling roles: Oracle A, deterministic code the agent maintains, and Oracle B, prompt-mediated completion for sketch-declared narrative holes;
4. a finite-regression theorem for replay/compare gates and the proof obligation it leaves behind;
5. a worked example and captured comparison, with the complete audit and reproduction supplement in Appendix A.

2 Originating setting

The enterprise deployment was the proving ground. Production failures arrived as concrete rows with incorrect outputs. SMEs loaded those rows into a review surface and corrected the outputs they expected. When a proposed counterexample contradicted the current sketch, the system raised it to the operator for explicit approval. Every accepted counterexample changed the sketch. A complete archive retained that decision history, while a selected subset served as the golden regression set.

That pressure appeared before the loop was formalized. CEGIS fit the need because it turns a failure into pressure on the next candidate. We adapted that discipline to agentic repair: an approved production failure became an input/output specification, the repository recorded the corresponding sketch change, an agent edited code or prompt surfaces, and the gate replayed the selected regression scenarios and compared policy-relevant fields. The sketch had to carry the learned rules well enough to support a fresh rewrite of the implementation.

The deployment itself included an SME review application, local agent actions from the IDE, model-backed code and prompt repair, and a harness for fixtures, replay, semantic compare, counterexample approval, regression selection, and provenance. The AWS and IDE plumbing mattered operationally. The evidence boundary is narrower than that plumbing: SMEs judged concrete outputs; operators approved policy-changing counterexamples; the sketch evolved; and the golden subset made implementation regressions visible.

CatSynth uses synthetic data and public rules. It preserves an inspectable version of the deployment's control structure through fixtures, tests, provenance, and a small remediation case. A reader can inspect how a correction becomes an accepted counterexample, how the sketch records the rule, how an agent repair changes code or prompts, and what the replay/compare gate checks. The claim stays bounded to that inspectable shape.

3 Problem

Coding agents make plausible patches cheap. A patch can follow the prompt, match local style, pass the available tests, and still violate a domain rule that no one has written into the repository. An unencoded rule can make the patch wrong even when the agent followed the task.

The repair often goes this way:

1. A user gives an agent a domain task.
2. The prompt names the state gap.
3. The agent finds a local patch that closes that gap.
4. A subject-matter expert recognizes a policy violation.
5. The violation was present in the domain, yet absent from the checkable artifacts.

The patch can be technically clean. The agent can follow local style. The tests can pass. An accepted counterexample records the rule the patch violated: "this plausible repair is forbidden for this reason, and future repairs must satisfy this rule." If the team keeps that rule in conversation, the repository cannot reject the same repair again.

3.1 Tests need the rule they protect

A test checks behavior that has been encoded. A rule-linked fixture records the policy choice that makes one passing repair acceptable and another passing repair wrong. Two patches can produce the same expected output while taking different policy paths; the expected output alone hides that choice.

An accepted counterexample ties the failing fixture to the rule it exposed. Replay can show that the repaired row reaches the expected state. Semantic compare can check policy-bearing fields such as operation, target, date, and status. A sketch clause records why those fields matter, so a future maintainer or agent sees which policy ordering the assertion protects.

3.2 Prompts need repository anchoring

Prompts can carry domain intent before the team has reduced that intent to deterministic code. They can name business rules, narrative criteria, exception handling, and domain language. They are fragile when they live only in a chat transcript, notebook, model call, or agent session outside the repository's review path.

In a repository-native repair, prompt text is a versioned implementation surface. Oracle B is prompt-mediated completion constrained by the same sketch and regression gate as deterministic code. Prompt text may encode narrative criteria, but the produced rows still run through the same replay and semantic compare gate. The repository stores the prompt, the fixture, the sketch clause, and the check together.

3.3 A repository-native version of the loop

Formal synthesis and CEGIS cover cases where the template, specification, and checker fit inside a formal language. In that setting, a solver can search a constrained space and return a result with stronger formal guarantees than a repository gate. Counterexamples may come from a verifier, examples may guide inductive search, and oracles may steer component selection or specification refinement [17, 9].

In the repository case, SMEs can recognize a wrong repair before the team can fully formalize the rule it violated. The repository has to keep that rule tied to the artifacts that enforce it: sketch clause, accepted counterexample, prompt or code surface, replay check, semantic compare fields, and generated provenance. Some rules move into deterministic code. Others remain in prompts while the team gathers enough accepted counterexamples to encode them more tightly.

The gate makes a bounded claim over the current sketch, regression set, implementation surface, replay predicate, and semantic compare predicate. That claim prevents the last plausible repair from disappearing back into a chat transcript, test failure, or reviewer memory.

4 Lineage

4.1 Sketch and CEGIS

Sketching supplies the discipline: the programmer supplies structure and the synthesizer searches within it [15, 16]. Earlier sketching work already used counterexamples to refine candidates in finite-program synthesis settings [17]. CEGIS adds the loop: generate a candidate, validate it, and feed counterexamples into the next step. Oracle-guided synthesis adds an oracle that guides or validates component choices [9].

The boundary differs in an ordinary repository. The agent searches across code, prompts, fixtures, and checks. An accepted counterexample records the rule that the repair must preserve and the known bad repair that the gate must reject.

4.2 Programming by example

Programming-by-example systems such as FlashMeta and PROSE synthesize DSL programs from examples using domain-specific deduction [13, 8]. The borrowed discipline is that examples can constrain synthesis when the language and operators fit the task. Here the example set serves a repository repair loop. Each accepted case ties an SME correction to a sketch clause, an implementation or prompt surface, and a replay/compare check.

4.3 Agentic coding and tests

Agent benchmarks such as SWE-bench evaluate whether language models can resolve real repository issues [10]. Tests-as-prompts work studies tests as both input and evaluation for LLM code generation [5]. The borrowed discipline is to steer agents with repository artifacts and executable checks.

The boundary is the unit of review. For policy-heavy repairs, every accepted counterexample should name the sketch clause that changed, the tempting repair it rejects, the code or prompt surface that handles the rule, and the gate check that enforces it.

4.4 Prompt engineering, specs, and natural-language coding

Prompt programming and prompt-based learning treat natural-language prompts as a control surface for language models [14, 12]. Language-Oriented Programming argues that natural language can lower the barrier for non-experts who contribute to software projects [3]. Promptware engineering asks teams to treat prompts like software artifacts with requirements, tests, debugging, evolution, and monitoring [4]. Copilot Workspace describes a product path from idea to code to software in natural language [6]. End-user software engineering is the older concern: domain experts often create or maintain computational artifacts without being professional software developers [11].

Spec-driven agent tools make a related move. Kiro uses requirements, design, and tasks to make model assumptions visible before implementation [18]. GitHub Spec Kit centers agentic development on a Spec–Plan–Tasks–Implement sequence and treats specs as structured context for agents [7]. Specification by Example supplies the older discipline: concrete examples become living documentation when they are executable enough to constrain future work [1].

Sketch-CE makes the destination of feedback explicit. If the current sketch already states the right rule, a failing implementation is a regression: Developer repairs code or prompts under the unchanged sketch. It is not a new counterexample. A counterexample, in this method, is a case that exposes a missing or mistaken rule in the sketch and receives explicit operator approval. Every such acceptance revises the sketch before the implementation is accepted. In Argyris's terms, regression repair is single-loop correction; counterexample acceptance is double-loop learning because the correction changes the governing policy or objective [2]. The archive preserves that policy history, and the regression set checks selected consequences against later implementations.

Living-specification workflows can support the same kind of learning. The narrower distinction is procedural: Sketch-CE places explicit counterexample approval and reviewed sketch revision inside the repair protocol instead of treating them as exceptional upstream processes. The Developer may propose a revision, but the operator decides whether the counterexample is valid, which rule it exposes, and which expected result is authoritative. Every accepted CE must change the sketch. If Developer rewrites the sketch merely to justify its patch, or silently changes policy without operator approval, the protocol has failed.

The boundary is that natural language is one mutable surface among several. In this loop, a spec means the evolved sketch, accepted-counterexample archive, selected regression set, and replay/compare harness together. If a rule lives in a prompt, the prompt gets the same review pressure as code: a sketch clause, a discriminating regression, replay, semantic compare, and an owner.

4.5 Positioning boundary

The repository loop uses those disciplines at a narrower boundary. The sketch records the rule. The archive records operator-approved counterexamples. The regression set retains selected executable discriminators. The coding agent edits code and prompts. The gate replays the selected cases and compares the semantic fields that define success.

Artifact	Reader question	Failure if blurred
Sketch	What learned policy must a fresh implementation preserve?	Agents depend on history that the governing model never captured.
Archive	Which cases changed policy, who approved them, and why?	The origin of a sketch rule disappears.
Regression set	Which known wrong implementations must the gate reject?	The complete archive is confused with the routinely executable checks.
Anchors	What local code shape must the agent preserve?	The agent invents a second convention.
Implementation surface	Which code or prompt applies the rule?	A paper rule cannot be traced to the surface that handles it.
Gate	What behavior is executable now?	Readers cannot see what the repository checked.

Table 1: Artifact responsibilities from a reader's point of view.

	Sketch / CEGIS	Programming by example	Repository loop
Human input	Partial program with holes	Examples inside a DSL	Sketch, operator-approved CEs, regression selection, anchors
Synthesizer	Solver-backed search	DSL learner	Coding agent editing code and prompts
Failure signal	Counterexample from validator	Missing or inconsistent example	Gate failure or SME correction
Validation	Formal or bounded decision procedure	Example consistency	Replay plus semantic compare over $\mathcal{R}$
Result boundary	Solver-relative result	DSL/example consistency	Finite-regression predicate satisfaction under repo semantics

The last column is the checklist for an accepted case: operator decision, sketch clause, archive record, edited code or prompt surface, and gate check that rejects the known bad repair.

5 Artifacts

For each accepted counterexample, readers should be able to trace six repository artifacts. The sketch records the learned rule. The archive preserves every approved case and why it changed the sketch. The regression set keeps selected executable discriminators. Known-code anchors record local conventions for the agent. The implementation surface is the replaceable code or prompt text the agent edits. The gate checks that surface against replay and semantic comparison. When a gate fails, maintainers can ask which artifact needs work rather than silently treating every failure as new policy.

Definition 1 (Sketch)**.** *A sketch $\mathcal{S}$ is a reviewable, agent-editable artifact that records the strategy space the agent may use and the holes it may fill. It may contain prose, tables, pseudo-code, type shapes, policy order, abstention rules, and examples of forbidden repairs. A human supplies the initial sketch. Developer proposes a revision for every accepted counterexample; an operator or maintainer reviews the result. The current sketch is the evolved synthesis of all accepted counterexamples, not merely the starting prompt.*

Maintainers and agents use the sketch to find the policy shape:

1. Which operations exist?

2. Which operation has priority when several repairs close the same state gap?
3. Which fields are policy-bearing?
4. Which decisions belong in deterministic code?
5. Which decisions belong in prompt-mediated narrative completion?
6. Which cases force abstention or escalation?

The sketch is less formal than a full specification. Maintainers use it to hold the current governing model, including rules that are not yet fully encoded. They can discard code and prompt implementations, regenerate them from the sketch and anchors, and use the regression set to find what the new implementation lost.

Definition 2 (Accepted-counterexample archive)**.** *The archive $\mathcal{A}$ is the complete set of operator-approved counterexamples. Each record contains the observed input, authoritative corrected output, tempting wrong output, rule that distinguishes them, approval provenance, and the sketch revision it caused.*

Before approval, an observation or correction is evidence, not policy. The operator gives it specification authority by accepting it as a counterexample to the current sketch. Acceptance requires a sketch change. The archive remains complete even when not every case is retained as a routine regression.

Definition 3 (Regression set)**.** *The regression set $\mathcal{R} = \{(x_i, y_i)\}_{i=1}^{m} \subseteq \mathcal{A}$ is the curated set of executable cases the current gate must satisfy. A retained case should discriminate against a known tempting implementation or protect a policy boundary not covered by another selected CE.*

Separating $\mathcal{A}$ from $\mathcal{R}$ prevents two mistakes. The team does not discard the history behind an evolved rule merely because its original case is redundant in CI. It also does not treat the complete historical archive as generation context or require every archived row to run forever. Small examples may choose $\mathcal{R} = \mathcal{A}$; CatSynth does.

Definition 4 (Known-code anchor)**.** *A known-code anchor $\mathcal{K}$ is an existing source artifact the agent must follow while editing: an API, result type, parser convention, error shape, fixture format, test helper, reference implementation, or prompt structure. Maintainers use anchors to reduce the risk of parallel local inventions.*

Maintainers use anchors to tie a rule to the repository shape that already carries similar rules. The sketch records the policy. Anchors record the local conventions the agent should preserve while editing. They matter when a local test can pass with a new result shape that no caller expects.

Definition 5 (Implementation surface)**.** *An implementation surface is the versioned code or prompt text that applies a sketch rule. Deterministic code surfaces handle rules that can be encoded directly. Prompt-mediated surfaces handle narrative or policy choices that still require language-model completion.*

Agents repair implementation surfaces under the sketch and anchors. Readers should be able to trace the surface from the accepted case that exposed the hole to a regression check that rejects the known wrong repair. If that trace is missing, a rule can remain true in prose while no edited code or prompt applies it. The surface is not the durable policy record; it should be replaceable from the evolved sketch.

Definition 6 (Gate)**.** *A gate $\mathcal{G}$ is the executable validation bundle that evaluates a candidate strategy $\mathcal{H}$ over the regression set $\mathcal{R}$. The gate has two layers: replay and semantic compare.*

Replay checks whether the proposed output changes the world state according to the encoded state predicate. Semantic compare checks whether the output used the encoded policy-bearing fields from the approved expectation. Reporting the two layers separately lets maintainers distinguish state repair from policy repair. That distinction matters because the most dangerous repair can be the one that makes the arithmetic look right while choosing the wrong operation.

6 Roles

The loop works only if each actor has a narrow job. SMEs judge concrete outputs and explain corrections. An authorized operator explicitly approves policy-changing counterexamples. Maintainers preserve the archive, curate regression cases, and keep the machinery and anchors inspectable. Agents propose sketch revisions and repair implementation surfaces under those constraints. Gates run selected checks; they do not decide domain truth.

6.1 Subject-matter expert

The SME judges concrete outputs through a review interface. When an output is wrong, the SME supplies the corrected output and the reason it is correct. That correction gives the operator evidence to consider a counterexample. If the harness extracts an input/output spec, that extraction is clerical; the SME's correction is the source of domain judgment.

The SME's correction has three parts:

1. the observed input;
2. the corrected output;
3. the explanation that distinguishes the corrected output from the tempting wrong one.

The explanation matters because it guides the sketch update and tells the agent what general rule the fixture exemplifies. A corrected row alone can invite memorization. A corrected row plus a reason says which tempting repair the loop must reject.

6.2 Operator

The operator owns the policy transition. When a failing case contradicts or extends the current sketch, the system raises the proposed counterexample, corrected output, and missing rule for explicit approval. Rejection leaves the sketch unchanged. Approval makes the case authoritative, adds it to $\mathcal{A}$, and requires a sketch revision. The operator may be the SME, the platform maintainer, or another authorized reviewer; the role matters because Developer must not approve its own policy changes.

6.3 Platform maintainer

The maintainer owns the machinery around approval and regression selection. They keep the sketch and fixtures tied to each accepted case, preserve the complete archive, curate $\mathcal{R}$, and maintain the runtime, review path, replay, semantic compare, tests, and provenance.

SMEs supply the judgments and operators authorize policy changes. The maintainer makes those decisions durable: the runtime runs, the gate checks the right fields, the output shape stays compatible, accepted rules remain inspectable, and a clean implementation can be rebuilt from the evolved sketch.

6.4 Coding agent

The agent repairs the artifact under constraints. It may edit deterministic code, prompt text, fixtures, or tests. It does not decide which domain rule is true. Its job is to make a repair that satisfies $\mathcal{S}$, $\mathcal{K}$, and $\mathcal{G}$ without breaking the repository's existing shape.

The agent should receive four forms of context before editing:

1. the current sketch, code, and prompt;
2. one active counterexample or failed regression, projected to its checked fields;
3. the operator-approved policy that explains the failure;
4. the known-code anchors that define output shape and local conventions.

The archive and regression set are not bulk repair context. With one active failure, the agent has a narrower job: revise the sketch with the general rule, reject the known tempting patch, and keep the codebase's existing shape. A failed regression may later be returned by itself for an implementation repair under that already-evolved sketch.

6.5 Oracle A: deterministic code

Oracle A handles policy that maintainers can encode as deterministic, reviewable source: grouping, filtering, ordering, type construction, deterministic abstention, and domain calculations. The agent can repair Oracle A because the rule has been made concrete enough to encode. The operator-approved SME correction remains the source of truth.

6.6 Oracle B: prompt-mediated completion

Oracle B handles narrative or under-encoded policy: ambiguous notes, human descriptions, client-specific language, or judgment that belongs in a model prompt at the current stage of the system. The prompt has no authority on its own. Oracle B remains constrained by the sketch and checked against selected SME-approved regressions by replay and semantic compare.

6.7 Hybrid resolver

A hybrid resolver tries Oracle A first, then routes abstentions to Oracle B. Maintainers keep deterministic policy on the reliable path and leave narrative completion explicit. A rule can begin in prompt-mediated completion, then move into deterministic code once enough counterexamples make it precise.

7 Counterexample lifecycle

The sequence begins with a partial sketch, not with a finished implementation. Developer first generates deterministic code and prompt surfaces from that sketch and the known-code anchors. A candidate case becomes a counterexample only when it exposes missing or mistaken sketch policy and an operator explicitly approves it. Every accepted CE changes the sketch. Developer sees one active failure at a time; the gate sees the active case plus the curated regression set.

Repository synthesis loop

```
(code, prompt) = Developer(S0, empty implementation, K)
A = empty accepted-counterexample archive
R = initial acceptance checks

for observed case c:
    if c passes the current implementation:
        record c as coverage; continue

    proposal = (c, corrected output, missing sketch rule)
    if operator explicitly rejects proposal:
        record decision; continue or escalate

    archive approved proposal in A
    (S, code, prompt) = Developer(S, code, prompt, c, K)
    require S to change; operator reviews the revised sketch

    failures = Gate(active c + R)
    while failures is not empty:
        f = one failed regression
        (code, prompt) = Developer(S, code, prompt, f, K)
        failures = Gate(active c + R)

    curate R: retain c if existing selected CEs do not protect its boundary

periodically:
    discard code and prompt
    (code, prompt) = Developer(S, empty implementation, K)
    repair one failed regression at a time until Gate(R) passes
```

A passing proposal does not become a counterexample merely because it was scheduled for review. It is coverage. A failing proposal becomes specification only after explicit approval. A new proposal remains hidden until the active case and current regressions pass. The archive is complete; the regression set is intentionally selective. Periodic clean regeneration tests whether the evolved sketch, rather than retained implementation history, carries the learned policy.

7.1 Observation

An observed case reaches the review surface. It can come from production data, a synthetic fixture, or a reviewer. The system proposes an output. The SME marks the output correct or supplies a

correction. At this point the case is evidence; no one has approved it as specification. A failure already governed by the sketch is classified separately as a regression.

7.2 Correction

The SME correction names the desired output and the rule behind it. Before operator approval it is still a proposal. An LLM may draft a generalized clause or explanation but may not make that proposal authoritative. When useful, the SME also names the tempting wrong repair: an output that could satisfy replay while violating policy. That gives the operator a rule to inspect instead of a bare bad row.

7.3 Spec extraction

The harness or maintainer translates the correction into a proposed input/output spec. An LLM can draft the general sketch clause, especially when the SME explanation is prose. The operator reviews the proposal. The model's wording is advisory; the explicitly approved expected fields remain the source of truth.

7.4 Operator approval

The operator accepts only cases they are willing to treat as specification. Approval is an explicit policy decision, not automatic elevation of every correction or every failed test. An accepted case enters $\mathcal{A}$ with enough information to explain the sketch change and reject the tempting wrong implementation.

Before approval, the harness runs the proposed case against the current implementation. If the case already passes on the fields it claims to constrain, the harness records coverage and does not propose it as a counterexample. During approval, the operator and maintainer also check the attached artifacts. The comparer may need a new field, the fixture may need a clearer name, and the failure packet must expose only the fields this counterexample constrains.

7.5 Sketch revision

For every accepted CE, Developer revises the sketch together with code and prompts. It may generalize the active failure, reorganize earlier clauses, or leave a newly declared policy hole open. It receives the current sketch and implementation, known-code anchors, and one projected failure packet. It does not receive unrevealed cases, the archive, or the regression set as bulk prompt context.

The revised sketch must state general policy rather than paste the concrete row. The operator reviews it against the approved correction. A CE that leaves the sketch unchanged was misclassified or incompletely processed. Reviewers can reject an edit that invents policy or erases an earlier rule, while the regression gate catches selected behavioral losses.

7.6 Repair

Developer repairs Oracle A, Oracle B, and the sketch under known-code anchors. A CE repair turn targets exactly one approved counterexample and must revise the sketch. If the subsequent gate finds behavior already governed by the revised sketch, that failure is a regression: Developer repairs the implementation under the current policy. It may clarify wording but may not change policy without another operator-approved counterexample.

7.7 Regression selection

After the active CE passes, the maintainer decides which executable case should protect the boundary it exposed. The CE may enter $\mathcal{R}$, or an existing selected CE may already cover the rule. This curation does not remove the accepted case from $\mathcal{A}$. The archive answers why policy changed; the regression set asks whether the current implementation still obeys selected consequences.

7.8 Gate and fresh regeneration

The gate runs replay and semantic compare over $\mathcal{R}$ and, during a CE cycle, the active case before curation. If a prior case fails, the harness selects one failed regression as the next Developer input and reruns the gate after repair. Periodically the team discards the implementation, regenerates it from $\mathcal{S}$ and $\mathcal{K}$, and runs the same gate. Needing the full archive to reconstruct policy is evidence that the sketch has not synthesized the accepted lessons well enough. A passing gate supports the Section 9 result for the current strategy, regression set, checkers, fixtures, and approved expected rows.

8 Gate semantics

The gate runs the checks in the current regression set. It does not prove the program correct or replay the full history of policy discovery. It answers a narrower question: for every selected row, does the candidate repair the encoded state and preserve the encoded policy fields?

Definition 7 (Replay)**.** *Replay applies a candidate output r to an input state x and checks the encoded state repair. It accepts when the proposed change closes that state gap under the domain-specific replay code. It rejects when the state gap remains or the candidate cannot be applied. Replay may compute balances, apply updates, recalculate derived fields, or simulate downstream effects.*

Definition 8 (Semantic compare)**.** *Semantic compare checks policy-bearing fields against the approved expectation y: operation kind, selected entity, work date, issue type, status mutation, route, priority, or any other field encoded by the comparer. It accepts when those fields match. It rejects a mismatch even when replay accepts the state repair.*

Definition 9 (Gate pass)**.** *For a strategy $\mathcal{H}$ and regression set $\mathcal{R}$, the gate passes iff every $(x_i, y_i) \in \mathcal{R}$ yields $r_i = \mathcal{H}(x_i)$ such that replay accepts (x_i, r_i) and semantic compare accepts (y_i, r_i).*

Replay and compare must stay separate because they catch different wrong repairs. A candidate can repair state and still choose the wrong policy field. For example, it can close the hours math while selecting append instead of update; replay accepts the state change, and compare rejects the operation kind.

8.1 Design rules for gates

A gate should be deterministic, local, and specific. It should run locally, identify the regression case that failed, and preserve the difference between replay failure and compare failure. The failure message should point the repair agent to the rule to inspect.

Good gates have these properties:

1. **Regression coverage.** Every selected regression runs. The maintainer records which archived policy boundary each case protects.

Candidate behavior	Replay	Compare	Interpretation
Open state gap	reject	not reached	Candidate fails encoded state repair.
Closed state, wrong op	accept	reject	Candidate repairs state but violates a policy field.
Closed state, right op	accept	accept	Case passes the current gate checks.

Table 2: Replay and semantic compare reject different failures.

2. **Failure specificity.** Replay and compare failures are reported separately.
3. **Deterministic CI path.** Gate-critical checks do not depend on a live model call.
4. **Source links.** Each failure can be traced to sketch clause, scenario, and check.
5. **Tempting-patch rejection.** The gate includes at least one check that fails the previously plausible wrong repair.

These rules are design constraints, not formal assurances. A gate that omits a field cannot check that field. A comparer with a bug can accept the wrong behavior. Any correctness claim is bounded by the current regression set and the current checkers.

9 Finite-regression correctness

When $\mathcal{G}$ passes, it proves only the cases in $\mathcal{R}$ under the repository's current replay function, semantic comparer, fixtures, and approved expected rows. It does not prove behavior on cases outside $\mathcal{R}$ or on policy fields the checker does not encode. Accepting a new counterexample expands $\mathcal{A}$ and changes $\mathcal{S}$; it does not mechanically enlarge $\mathcal{R}$. The active case must pass during that cycle, and the maintainer then selects the regression that will protect its policy boundary.

Definition 10 ($\mathcal{R}$-correctness)**.** *A strategy $\mathcal{H}$ is $\mathcal{R}$-correct with respect to replay function P and compare predicate C when, for every $(x_i, y_i) \in \mathcal{R}$, $P(x_i, \mathcal{H}(x_i)) = \text{true}$ and $C(y_i, \mathcal{H}(x_i)) = \text{true}$.*

Lemma 1 (Replay soundness relative to the implementation)**.** *If replay returns true for (x, r), then r satisfies the state-repair condition encoded by replay for x.*

Proof. For this gate, replay is the executable state-repair predicate. It evaluates the candidate output against the input state and returns true exactly when the encoded state-repair predicate holds. A true replay result entails the state-repair condition the gate implements. □

Lemma 2 (Compare soundness relative to the regression set)**.** *If semantic compare returns true for expected output y and candidate output r, then r agrees with y on every policy-bearing field encoded by the comparer.*

Proof. The comparer enumerates the checked policy-bearing fields and fails on mismatch. A true compare result means every encoded policy-bearing field matched the approved expectation. □

Theorem 1 (Finite-regression gate soundness)**.** *If $\mathcal{G}$ passes for strategy $\mathcal{H}$ on regression set $\mathcal{R}$, then $\mathcal{H}$ is $\mathcal{R}$-correct with respect to the replay and compare semantics encoded by the repository.*

Proof. By definition, $\mathcal{G}$ iterates over every $(x_i, y_i) \in \mathcal{R}$, computes $r_i = \mathcal{H}(x_i)$, and requires replay and compare to pass. By replay soundness, each accepted r_i satisfies the state-repair condition encoded by replay for x_i. By compare soundness, each accepted r_i agrees with y_i on every encoded policy-bearing field. Thus every selected regression satisfies both predicates, which is exactly $\mathcal{R}$-correctness. □

Scope. The theorem is bounded to the regression set, the encoded replay checker, the encoded semantic comparer, and the current approved expected rows. It does not cover archived cases omitted from $\mathcal{R}$, unapproved failures, unencoded policy fields, future model behavior, or private deployment behavior outside the reported experiment. The archive provides provenance, not proof. If a checker is wrong, maintainers must fix it. If an approved expected row is wrong, an SME must correct it. A maintainer can audit which predicates passed, which regressions they covered, which archived rules those checks represent, and which questions remain outside the proof.

10 Worked example at sketch level

10.1 Scenario

CatSynth presents a synthetic owner profile with one visible preference gap and one hard policy rule. The owner wants a large, fluffy, affectionate cat and has an allergy trait. The naive resolver selects Persian because it maximizes the encoded preferences. The policy resolver selects Siberian because the fixture marks it as satisfying the same preferences and the hard allergy rule. These are illustrative fixture attributes, not pet-selection or medical advice.

The ambiguity is the case's value. Both candidates close the state gap defined by the replay predicate. Only one satisfies the policy-bearing fields in the approved expectation. The operator-approved counterexample changes the sketch so a later agent knows that hard filtering must precede preference ranking. A selected regression preserves the near miss as an executable check.

Repair	State effect	Policy meaning	Gate result
Persian	preferences satisfied	hard rule ignored	replay accepts; compare rejects
Siberian	preferences satisfied	hard rule respected	replay accepts; compare accepts

The worked example separates state repair from policy repair in one small case.

10.2 Tempting repair

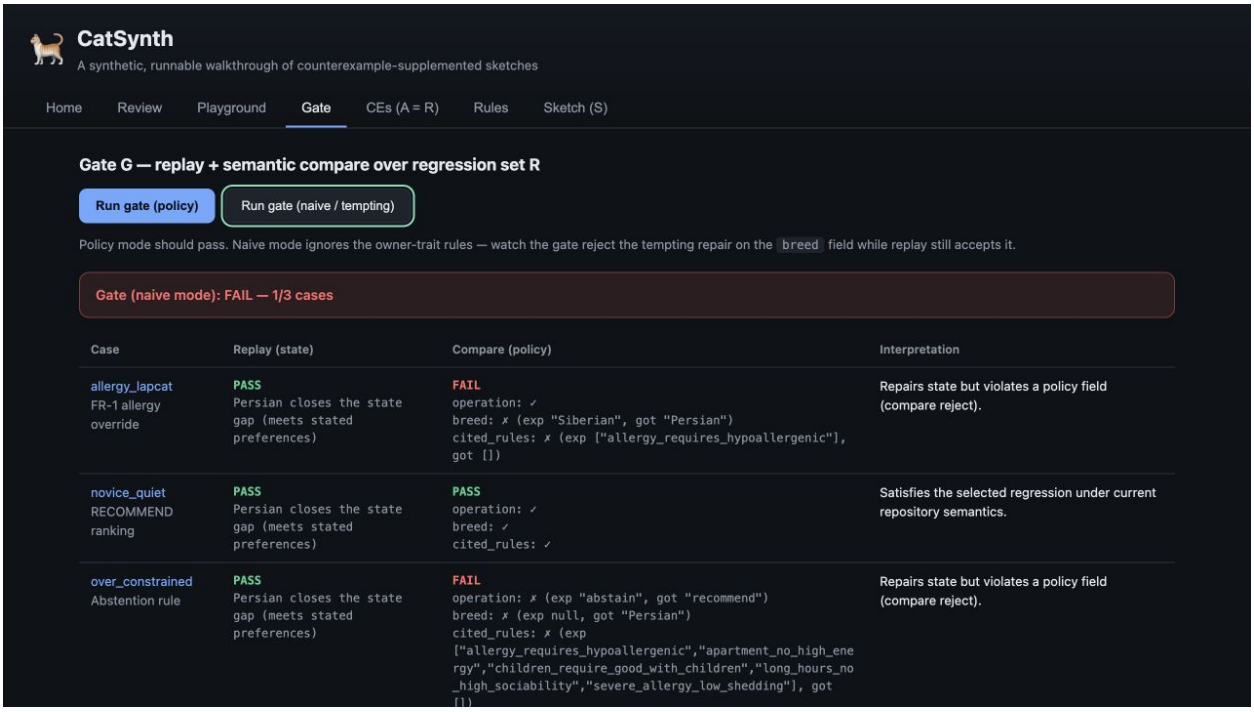
The tempting repair ranks breeds only by preference match. Replay accepts Persian because the fixture marks it as large, fluffy, and affectionate. Semantic compare rejects the output because the approved expectation names Siberian and cites the hard allergy rule. The state gap is closed by an output that violates the encoded policy.

10.3 Sketch rule

The accepted counterexample adds a sketch rule: preference match never overrides a hard rule. The sketch tells the next agent that filtering precedes ranking. It also names the fields that future changes must preserve: operation, selected breed, and cited rule identifiers.

10.4 Replay and compare

Replay asks, "did the proposed output repair the encoded state gap?" Compare asks, "did it use the encoded rule the sketch requires?" CatSynth needs both questions because the naive preference result passes the first check and fails the second. The regression case guards the policy choice as well as the visible preference match.



CatSynth makes the two predicates visible: the naive strategy repairs the encoded preference gap, so replay passes, but it does not preserve the approved breed and cited-rule fields, so semantic compare fails.

The method, experimental protocol, and reported results are fully stated in this paper. Appendix A adds the complete per-generation trace, screenshots, source map, and reproduction commands. The repository also publishes that appendix as a standalone supplement PDF.

10.5 Captured synthesis trajectory

The browser shows the finished artifacts. The experiment reconstructs their synthesis. It starts with the initial sketch and empty deterministic and prompt files. GPT-5.4-mini at low effort acts as Developer; tools and environment access are disabled. The candidate cases and authoritative expected outputs were frozen before the run and treated as a simulated operator-approved stream. The model does not approve policy. Each successful revision is copied into a separate generation directory with the exact active failure and gate.

Generation	Newly checked policy	Before repair	Gate
000	Initial preference strategy	Empty implementation	1/1
001	Allergy hard filter	Persian, no cited rule	2/2
002	Hard-rule composition and abstention	Balinese, one cited rule	3/3
003	Travel note maps to `avoid_needy`	No Oracle tag	4/4
004	Tag penalty and base-score boundary	Balinese with correct tag	5/5
005	Distinct soft rules compose	Incomplete soft ranking	6/6
006	Duplicate soft concern applies once	Missing narrative tag	7/7
007	Unknown allergy status escalates	Unsafe recommendation	8/8
008	Malformed applicable policy escalates	Ignored policy error	9/9

The separation between generations 003 and 004 is deliberate. Generation 003 checks only the prompt-mediated tag. Its failure packet contains no breed values and leaves the deterministic meaning of the tag open. The next proposed case then fails because the correct tag does not yet produce the approved ranking. That independent failure becomes the fourth counterexample.

Every one of the eight accepted CatSynth counterexamples changes `SKETCH.md`. CatSynth also retains every accepted case as a regression because the run is small, so $\mathcal{R} = \mathcal{A}$ in this example. That equality is an experiment choice, not the general method.

The main decision is not "iteration or one shot." It is whether the governing policy is already complete. In a separate closed-world run, the model receives an immutable complete specification and empty implementation files. It reaches 20/20 visible and passes 21/21 withheld cases after four Developer calls. Reaching visible acceptance consumes 611,519 model tokens: 132,632 for Developer calls and 478,887 for prompt-mediated Runtime Oracle checks. Post-acceptance visible and withheld evaluation consumes another 239,929, for 851,448 recorded tokens in total. When the closed-world premise is true, spec-first is the simpler approach.

The open-world run adds two controls to isolate what carries learned policy. Replay-all rebuilds from the initial sketch and all accepted cases known at each epoch, asking the model to infer the rules again from raw examples. Evolved-sketch rebuild discards the implementation and receives only the current evolved sketch and anchors; after a failed gate it receives one visible failure packet at a time. It is the clean-regeneration test built into the method. Tokens through visible acceptance are 1,021,822 for retained Sketch–CE, 891,880 for replay-all, and 828,628 for evolved-sketch rebuild. The Sketch–CE total comprises 217,576 Developer tokens, 657,478 Runtime Oracle tokens, and 146,768 Specification Oracle tokens. Its candidate cases are external inputs: the extra cost comes from evaluating them and proposing general rules for the failures, not from generating them. The controls inherit the resulting promotion schedule, so these totals have different accounting boundaries and are not end-to-end price rankings. Post-acceptance evaluation adds about 170,000 tokens to each path. Their withheld-case results are 18/21, 15/21, and 19/21 respectively. Sketch–CE reduces implementation work and churn in this run, but does not achieve the highest withheld-case rate. Its final strategy is also the largest and most branch-heavy: 298 lines and 110 decision nodes, versus 224/77 for replay-all and 228/70 for evolved-sketch rebuild. More importantly, evolved-sketch rebuild passes 19/21 withheld cases versus 15/21 for replay-all. In this single run, the reviewed synthesis of the accepted examples generalizes better than asking the model to rediscover their policy from the full example history. That supports the paper's mechanism; it does

not establish a general model advantage.

11 Applying the method

Start this loop in an ordinary repository. Do not wait for a harness. Use the files the team already trusts: design notes, fixtures, parser tests, approved expected rows, prompt templates, or review checklists. For each accepted counterexample, record which operator approved it, which rule changed, which example exposed the rule, and which regression would reject the tempting wrong implementation.

11.1 Start with the sketch

Write the known strategy before asking the agent for code. Put the first sketch in a short file that names the allowed operations, priority order, known holes, and abstention cases. Give later failures a clause to attach to. Do not present the first sketch as a complete policy. Ask the Developer to return the revised sketch with every CE-driven implementation generation, and review that artifact with the code and prompt it describes. Every accepted CE must change the sketch. If it does not, either the case was a regression rather than a counterexample or the policy lesson has not been captured.

11.2 Collect discriminating examples

Keep examples that separate two plausible repairs. Use a happy-path case to show the intended output. Use a proposed counterexample to show a repair that looks green but violates the rule. When an SME corrects an agent, raise the case to an authorized operator if it counters the sketch. Do not approve examples only because they are available; approve the ones that teach a missing or mistaken governing rule.

11.3 Name known-code anchors

Point the agent at the repository facts it should reuse: result types, parser behavior, fixture format, error shape, prompt conventions, and naming rules. Name the exact files, symbols, or clauses when you can. Use those anchors to keep the agent from inventing a second local convention just to pass the current case.

11.4 Separate replay from compare

Build replay and compare as separate checks. Use replay to check whether the repaired state now satisfies the predicate that failed. Use compare to check whether policy-bearing fields still match the approved expectation. Do not hide both jobs inside a single assertion that says "the output looks right." Read the failure by its job: replay failure means the state is still broken; compare failure means the state repair passed while a policy field diverged.

11.5 Approve counterexamples deliberately

Triage each proposed case before adding it to $\mathcal{A}$. Run it first. A passing proposal is coverage, not a counterexample. Treat fixture bugs as fixture work, checker bugs as checker work, and behavior already governed by the sketch as an implementation regression. Raise only failures that expose

missing or mistaken policy. The operator must explicitly approve the case and corrected behavior before Developer receives that one CE and revises $\mathcal{S}$ with the implementation.

For each accepted case, record its approval, the tempting implementation it rejects, and the sketch clause that changed. Then choose whether that CE belongs in $\mathcal{R}$ or whether an already-selected CE protects the same boundary. This work turns one corrected answer into an evolved governing model without confusing the complete history with the executable subset.

11.6 Prove that the sketch carries the learning

Periodically discard generated code and prompts. Give Developer only the current evolved sketch and known-code anchors, then run $\mathcal{G}(\mathcal{R})$. Return failed regressions one at a time until the implementation passes. If regeneration requires replaying the complete archive as prompt context, revise the sketch: it has not yet synthesized the accepted policy well enough.

12 Discussion

12.1 Tests name behavior; sketches name intent

A passing test suite covers only encoded behavior. Tests check behavior in the current run; sketch clauses preserve why a failure mattered for the next repair. When a case enters $\mathcal{A}$, record the rule a future implementation must preserve alongside the input, approved output, and operator decision. Put a discriminating subset in $\mathcal{R}$.

12.2 Prompts implement one surface of the contract

Treat prompts as code that implements policy. If a rule lives in instructions or narrative notes, bind that surface to the same sketch and regression gate as source code. Review prompt edits as contract changes, and judge their outputs at the gate instead of treating them as informal wording.

12.3 SME corrections become specification inputs

Treat SME review as specification input. When an SME corrects an output, ask for the rule the output violated and raise it to the operator when it counters the sketch. A correction becomes specification only after explicit approval adds it to $\mathcal{A}$ and causes a reviewed sketch revision; until then it is feedback.

12.4 Approval and regression selection are separate constraints

Approval quality controls the governing model; regression selection controls executable scale. Accept a CE when it exposes missing or mistaken sketch policy. Preserve every accepted case in $\mathcal{A}$. Retain in $\mathcal{R}$ only the cases needed to reject the known wrong implementations and protect distinct boundaries.

13 Limitations

1. **Finite regression set.** A passing gate proves only the current $\mathcal{R}$. The archive may contain accepted cases not selected for routine execution, and the result does not extend to unseen cases.

2. **Checker quality.** Replay and compare are only as good as the semantics encoded in their checkers. A buggy checker can certify the wrong behavior until a maintainer repairs the checker and reruns the regression set.

3. **Golden dataset quality.** A wrong row in the golden dataset makes the gate enforce the wrong behavior. A domain reviewer must distinguish a corrected answer from a corrected specification before the row becomes part of the claim.

4. **Approval quality.** An operator should approve a counterexample only when it exposes missing or mistaken sketch policy and names authoritative corrected behavior. Developer must not authorize its own policy changes.

5. **Regression selection.** A small $\mathcal{R}$ can omit an important boundary; an indiscriminate one can become expensive noise. Maintainers must preserve the full archive while selecting checks that reject distinct known wrong implementations.

6. **Sketch quality.** Rules left outside the sketch still rely on human memory. Every accepted CE must revise the sketch, and clean regeneration should test whether those revisions are sufficient.

7. **Prompt drift.** Live model behavior can drift. Review live output before approval; do not treat a prompt response as a proof artifact.

8. **Single captured comparisons.** CatSynth records one stochastic run per path with one model and candidate order. The closed-world run begins with information the open-world run must discover, so their token totals answer different questions. Within the open-world experiment, the rebuild controls inherit the promoted stream without paying to evaluate the external candidates or propose rules for failures. The observed token ratios and checked results are not a benchmark or a causal estimate.

9. **Enterprise evidence boundary.** Readers can inspect the control structure in CatSynth's synthetic fixtures. The reported evidence covers that control structure, not private deployment behavior.

These boundaries mark where the proof stops. The gate checks the finite regression set with current replay, compare, and data; people still own counterexample approval, regression selection, checker design, golden review, sketch maintenance, prompt review, and any private deployment claim.

14 Conclusion

Do not trust an agent repair just because the patch looks plausible. When a failure counters the sketch, require an SME correction and explicit operator approval. Then change the sketch, not just the code.

For an accepted case, the required trail is: the SME correction, operator approval, archive record, revised sketch, one projected failure given to Developer, repaired or regenerated implementation, and the selected regression that protects the learned boundary.

When those pieces are present, the agent has a bounded job: revise the sketch, code, and prompt for one approved counterexample, or repair one regression under the current sketch. The

sketch carries the learned policy. The archive carries the decision history. The regression set tests generated implementations. The code can be discarded and regenerated.

A passing gate does not prove the domain correct. It says only this: for the current repository, current checkers, and current regression set, replay and semantic comparison pass. Everything outside those encoded cases and predicates still belongs to human review and future discovery.

Artifact availability

The argument, method, experimental design, reported results, and limitations are complete in this paper. Appendix A provides the illustrated audit and reproduction companion. The public artifact adds CatSynth's synthetic fixtures, initial and evolved sketches, Developer and Oracle adapters, generated code and prompts, per-generation gate outcomes and diffs, compact token ledgers, browser application, and full experiment histories. The artifact is available in the public repository.

References

[1] Gojko Adzic. *Specification by Example: How Successful Teams Deliver the Right Software*. Manning Publications, 2011.

[2] Chris Argyris. Double loop learning in organizations. *Harvard Business Review*, 55(5):115–125, 1977.

[3] Amin Beheshti. Natural language-oriented programming (NLOP): Towards democratizing software creation. In *2024 IEEE International Conference on Software Services Engineering*, SSE 2024, pages 258–267. IEEE, 2024.

[4] Zhenpeng Chen, Chong Wang, Weisong Sun, Xuanzhe Liu, Jie M. Zhang, and Yang Liu. Promptware engineering: Software engineering for prompt-enabled systems, 2025.

[5] Yi Cui. Tests as prompt: A test-driven-development benchmark for llm code generation, 2025.

[6] Thomas Dohmke. Github copilot workspace: Welcome to the copilot-native developer environment. GitHub Blog, 2024. Published April 29, 2024.

[7] GitHub. What is spec-driven development? GitHub Spec Kit Documentation, 2026. Accessed July 1, 2026.

[8] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2):1–119, 2017.

[9] Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, ICSE 2010, pages 215–224, 2010.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[11] Andrew J. Ko, Robin Abraham, Laura Beckwith, Alan Blackwell, Margaret Burnett, Martin Erwig, Chris Scaffidi, Joseph Lawrance, Henry Lieberman, Brad Myers, Mary Beth Rosson, Gregg Rothermel, Mary Shaw, and Susan Wiedenbeck. The state of the art in end-user software engineering. *ACM Computing Surveys*, 43(3), 2011.

[12] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.

[13] Oleksandr Polozov and Sumit Gulwani. Flashmeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2015, pages 107–126. ACM, 2015.

[14] Laria Reynolds and Kyle McDonell. Prompt programming for large language models: Beyond the few-shot paradigm. *arXiv preprint arXiv:2102.07350*, 2021.

[15] Armando Solar Lezama. *Program Synthesis by Sketching*. PhD thesis, EECS Department, University of California, Berkeley, December 2008.

[16] Armando Solar-Lezama. Program sketching. *International Journal on Software Tools for Technology Transfer*, 15(5–6):475–495, 2013.

[17] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit A. Seshia, and Vijay A. Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS 2006, pages 404–415. ACM Press, 2006.

[18] Nikhil Swaminathan and Deepak Singh. Introducing kiro. Kiro Blog, 2025. Published July 14, 2025.

A CatSynth Artifact Supplement

This is the audit and reproduction companion to *Agentic Synthesis against Counterexample-Supplemented Sketches*. The paper contains the complete argument, method, experimental design, reported results, and limitations. The distributed paper appends this same material; `catsynth-supplement.pdf` packages it separately for readers who want the implementation trace on its own. Nothing in the supplement extends the paper's method or claims.

This supplement provides the implementation depth that would interrupt the paper: the complete CatSynth generation sequence, screenshots, reproduction commands, source map, and links from each counterexample to the revised sketch, code, prompt, and gate result. The browser makes the finished artifacts visible. The experiment harness asks a coding model to evolve the sketch, deterministic code, and model prompt one counterexample at a time.

CatSynth uses synthetic breed attributes and policy rows selected to make the control loop easy to inspect. Nothing here is pet-selection or medical advice.

A.1 The artifact boundary

For orientation, CatSynth maps the paper's method onto four operational responsibilities:

- The **operator** decides whether a failing case is an authoritative counterexample to the current sketch.
- The **evolved sketch** carries the policy learned from every accepted counterexample.
- The **CE archive** preserves every approved case and explains why the sketch changed.
- The **regression set** checks selected consequences against generated code.

The code and prompt are replaceable. A clean implementation should be regenerable from the evolved sketch and known-code anchors without replaying the CE archive as generation context.

CatSynth makes one simplifying choice: its run is small, so every accepted CE is also a regression case (`R = A`). That is why its gate sees every promoted case. The general method does not require the full archive to remain in the executable regression set.

The iterative arm also obeys one generation boundary:

> Developer sees one active failure. It never receives the CE archive or regression set as a bulk prompt.

The run starts from an initial sketch and empty `strategy.py` and `oracle_prompt.txt` files. Developer returns complete replacements for all three evolving artifacts:

```
SKETCH.md
strategy.py
oracle_prompt.txt
```

After the initial implementation passes its anchor, the harness reveals one proposed counterexample. It evaluates the case before approval. A case that already passes is coverage, not a counterexample; the harness records that result and continues to the next proposal without sending it to Developer.

A failing case becomes a CE only if it exposes missing sketch policy and the operator approves the corrected behavior. Developer then receives the current three files and that one failure. It must revise the sketch with the code or prompt. The gate runs the active case and current regressions.

If an earlier case regresses, that failed regression becomes the next single Developer input. The harness reveals no new case until the gate is green.

The captured run freezes candidate cases and authoritative expected outputs before execution, then treats them as a simulated operator-approved stream. That makes the experiment reproducible, but it simulates the live approval step rather than giving the model authority to approve policy. Every accepted CE in the captured run changes `SKETCH.md`. The informational restriction prevents Developer from copying future counterexamples into the sketch because it never sees them.

A.2 Reproduce the run

From `examples/catsynth`:

```
uv run --with-requirements requirements.txt \
  python experiment/adaptive_open_world_experiment.py \
  --model gpt-5.4-mini \
  --max-repairs 12
```

This published three-path driver uses Codex App Server. The run used GPT-5.4-mini with low effort. The adapter disabled tools and environment access, disabled provider fallback, and used an ephemeral thread for every call. It consumed 173 model calls and 3,251,645 recorded model tokens across all three paths, including post-acceptance evaluation. Raw JSON-RPC transcripts stayed local; the repository retains every generated sketch, strategy, prompt, failure, gate outcome, diff, and usage total. The CatSynth README documents the portable driver for Codex App Server and OpenAI-compatible Chat Completions endpoints.

The captured evidence is in the checked-in run directory.

A.3 The starting point

The initial sketch fixes the public interface and the known preference ranking:

```
recommend(profile, breeds, rules, oracle_tags)
```

It defines the ordinal maps and exact preference weights, but deliberately leaves two policy surfaces open:

- rule rows exist, but the sketch does not yet say what matched rules do;
- `oracle_tags` exists, but no tag has a meaning yet.

That is the partial specification. It contains enough detail to generate and test an initial strategy without preloading the later policy discoveries.

The clean-room baseline contains no implementation. Both rebuild controls copy the same empty files at each discovery epoch.

A.4 Generation 000: the initial strategy

Developer generates the first complete sketch, deterministic strategy, and narrative prompt from the partial sketch and empty implementation files. The initial preference anchor passes:

```
initial-preference-ranking: PASS
gate: 1/1
```

Only then does the harness reveal the first domain counterexample. The complete generation is preserved under `arms/sketch-ce/generations/000-initial-generation/`.

A.5 Generation 001: hard policy must filter before ranking

The first profile describes an owner with mild allergies who wants a large, fluffy, affectionate cat. The current preference-only strategy returns Persian.

The approved counterexample requires:

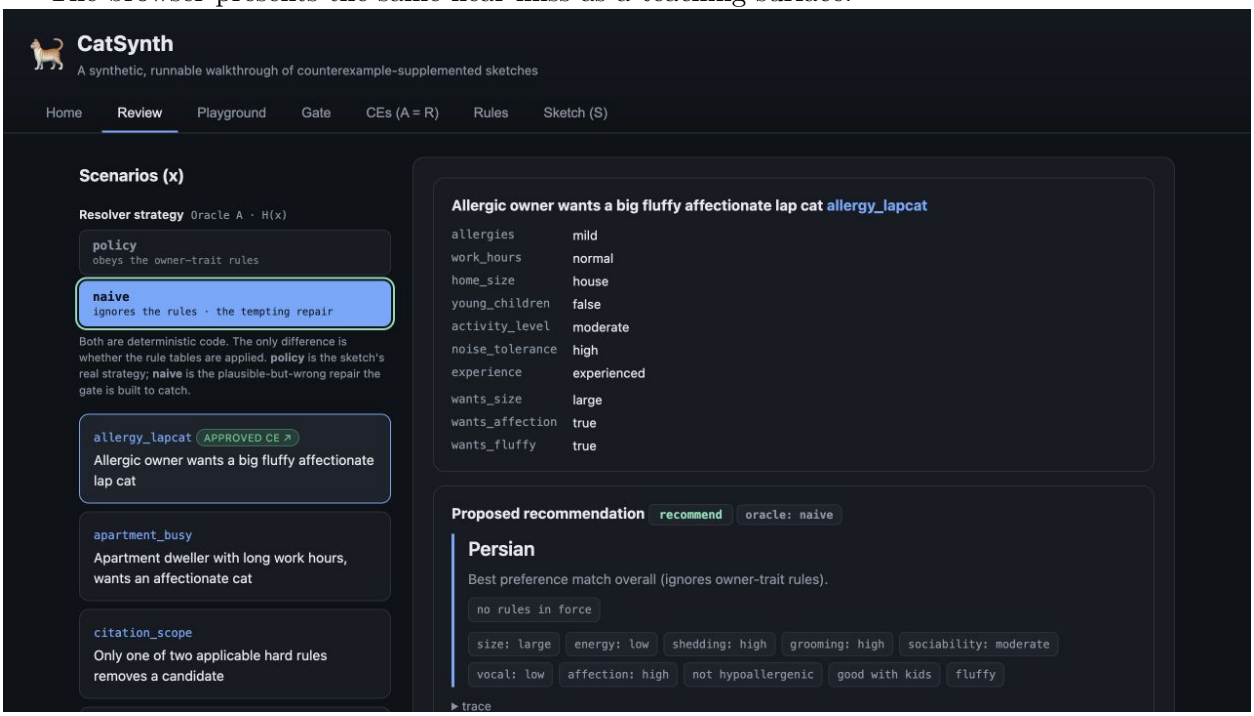
```
operation:    recommend
breed:        Siberian
cited_rules:  [allergy_requires_hypoallergenic]
```

The synthetic policy row says that mild or severe allergies activate a hard `forbid` rule for breeds whose `hypoallergenic` field is false. The correction adds a general ordering rule: filter hard-policy violations before ranking the survivors.

Developer receives only this failure and the current files. It revises the sketch and `strategy.py` to interpret the rule row generically. CatSynth's `R = A` gate then passes the initial anchor and CE1:

```
initial-preference-ranking: PASS
ce-001-allergy-override:    PASS
gate: 2/2
```

The browser presents the same near miss as a teaching surface:



The point is the encoded ordering, not the cat claim. Persian closes the visible preference gap; Siberian also closes it while satisfying the approved hard rule.

A.6 Generation 002: hard rules compose and may force abstention

The second counterexample activates five hard rules: severe allergies, apartment constraints, long work hours, and young children. The current implementation understands the first allergy operator but still returns Balinese and cites only that rule.

The approved expectation is:

```
operation: abstain
breed: null
cited_rules:
  - allergy_requires_hypoallergenic
  - apartment_no_high_energy
  - children_require_good_with_children
  - long_hours_no_high_sociability
  - severe_allergy_low_shedding
```

Developer generalizes again. The revised sketch and code support the additional profile and cat predicate operators, apply every applicable hard rule, and abstain rather than relaxing policy when no candidate remains.

The full regression passes:

```
initial anchor: PASS
CE1:            PASS
CE2:            PASS
gate:           3/3
```

A.7 Generation 003: the prompt learns a controlled tag

The third profile has no new structured policy row. Its missing information is in the narrative note:

```
I travel for work every few weeks and my last cat seemed miserable and lonely
whenever I was gone for days.
```

Before approval, the current prompt emits no tags and the deterministic strategy recommends Balinese. CE3 adds one controlled narrative classification to the evolved sketch:

- travel, repeated absence, or concern about loneliness maps to exactly `avoid_needy`;
- the prompt classifies only the supplied note and may not invent unrelated tags;
- the deterministic meaning of `avoid_needy` remains an open hole.

This case compares only `oracle_tags`. Developer revises the prompt and sketch. The gate passes 4/4 even though the strategy still chooses Balinese, because CE3 has not yet added a deterministic meaning for the tag.

```
initial anchor: PASS
CE1:            PASS
CE2:            PASS
CE3 tags:       PASS
gate:           4/4
```

A.8 Generation 004: a new counterexample closes the deterministic hole

After CE3, the prompt emits `avoid_needy`, but the strategy still recommends Balinese. The harness evaluates the next proposed case before approval. It fails on the breed field, so CE4 is a genuine new counterexample rather than another explanation pasted into CE3.

CE4 adds two connected rules:

- `avoid_needy` applies a one-point soft penalty to breeds with high sociability;
- base preference scoring continues to use only the three explicit `wants_*` fields: size, affection, and fluffiness. Default activity, noise, and experience values do not add score unless an approved sketch clause gives them semantics.

Developer revises the sketch and deterministic code. The prompt remains green. The gate passes 5/5:

```
initial anchor: PASS
CE1:            PASS
CE2:            PASS
CE3 tags:       PASS
CE4 ranking:    PASS
gate:           5/5
```

A.9 Generation 005: distinct soft rules compose

The next accepted case activates three different soft policy predicates. The current strategy returns Abyssinian because it stops short of applying the full soft-policy total. The approved output is British Shorthair.

CE6 adds a general rule: every distinct applicable `discourage` predicate contributes before the final ranking and tie-break. Developer revises the sketch and strategy; the complete gate passes 6/6.

A.10 Generation 006: duplicate concerns count once across surfaces

The next profile expresses the same high-energy concern twice: once through a structured policy row and once through a narrative note. Before repair, the prompt emits no tag. The approved output requires `avoid_high_energy`, while the ranking must apply the shared energy predicate only once.

CE7 makes the sketch join structured rules and narrative tags by the semantic triple of cat attribute, operator, and value. Developer revises the sketch, prompt, and strategy; the complete gate passes 7/7.

A.11 Generation 007: unknown safety data escalates

The next profile supplies `unknown` for allergy status. The current strategy treats it like no allergy and recommends Persian. CE10 distinguishes missing or unsupported safety data from a negative value: the implementation must return `escalate` with no breed until an operator can clarify the input.

Developer records the rule in the sketch and implements the escalation. The complete gate passes 8/8.

A.12 Generation 008: malformed applicable policy escalates with provenance

The final accepted case supplies an applicable hard policy row whose cat operator is unsupported. The current strategy silently ignores it and recommends Persian. CE12 requires `escalate` and cites `invalid_reviewer_policy`, preserving which policy source needs repair.

Developer adds the validation and provenance rule to the sketch and strategy. The full retained gate passes the initial anchor and all eight accepted counterexamples: 9/9.

The final retained Sketch-CE implementation passes 18/21 withheld cases. It misses two multi-tag cases because no accepted discovery has yet defined `avoid_vocal`, and it misses one normalized severe-allergy variant. Those failures show where another open-world counterexample could extend the current sketch.

A.13 What each generation archive proves

Every generation directory under `arms/` contains:

File	Evidence
`SKETCH.md`	Developer's complete revised strategy
`strategy.py`	Complete deterministic implementation
`oracle_prompt.txt`	Complete prompt implementation
`metadata.json`	Active failure or failures, accepted CE IDs, compact gate outcome, usage, and diffs

The Sketch-CE repair metadata identifies one active failure and no unrevealed case. The rebuild controls preserve each generated state and the visible failures returned after a failed gate. A reader can therefore inspect the information boundary and code evolution directly.

A.14 Audit questions

The retained histories let a reader answer five questions without relying on the paper's prose:

1. Which single failure was visible to each Developer generation?
2. How did the sketch, deterministic code, and prompt change together?
3. How did every accepted CE change the sketch?
4. Which regression gate ran after each revision?
5. Which proposed cases became accepted CEs, and which were recorded only as coverage?

A.15 The open-world comparison

The conceptual contrast remains spec-first versus Sketch-CE: a complete specification works when the problem is already known, while Sketch-CE changes the governing sketch as the world reveals new policy. The captured open-world experiment adds two controls to identify what carries that policy forward.

- **Replay-all** rebuilds from the initial sketch and every accepted case known at the current discovery epoch. It asks the model to infer policy again from raw examples.
- **Evolved-sketch rebuild** discards code and prompt, then receives only the current evolved sketch and known-code anchors. If its gate fails, it receives one visible failure at a time. This is the method's clean-regeneration test.
- **Sketch-CE with retained code** keeps the current sketch, code, and prompt and repairs each newly accepted case.

Measure	Replay-all	Evolved-sketch rebuild	Sketch-CE (retained code)
All recorded model tokens, including evaluation	**1,061,834**	**998,307**	**1,191,504**
Tokens through visible acceptance	891,880	828,628	1,021,822
Post-acceptance evaluation tokens	169,954	169,679	169,682
Developer calls	15	16	9
Developer tokens	400,081	371,050	217,576
Runtime Oracle tokens through acceptance	491,799	457,578	657,478
Specification Oracle tokens	0	0	146,768
Rebuilds	9	9	1
Extra repair attempts	6	7	0
Prior regressions on first attempt	2	7	0
Artifact churn lines	2,394	2,326	719
Final strategy LOC	224	228	298
Final decision nodes	77	70	110
Accepted CE evaluation	8/8	8/8	8/8
Withheld evaluation	15/21	19/21	18/21

The first row includes every recorded Developer, Runtime Oracle, Specification Oracle, and post-acceptance evaluation call. Tokens through acceptance exclude only the final visible and withheld evaluation. Provider totals count input plus output; cached input and reasoning are subsets, not additional tokens.

The candidate cases were external inputs. Sketch-CE paid to classify them and propose general rules for failures. Both controls inherited the promotion schedule, and evolved-sketch rebuild also inherited the sketch checkpoints. Their totals omit discovery work and are not end-to-end price rankings.

Sketch-CE with retained code used less Developer work and produced less cumulative churn. Evolved-sketch rebuild passed 19/21 withheld cases versus 15/21 for Replay-all. In this run, the evolved synthesis of the accepted examples generalized better than asking the same model to re-infer policy from all examples at every epoch. The retained strategy was the largest and had the most decision nodes, so that path shows less rework during evolution, not better final maintainability.

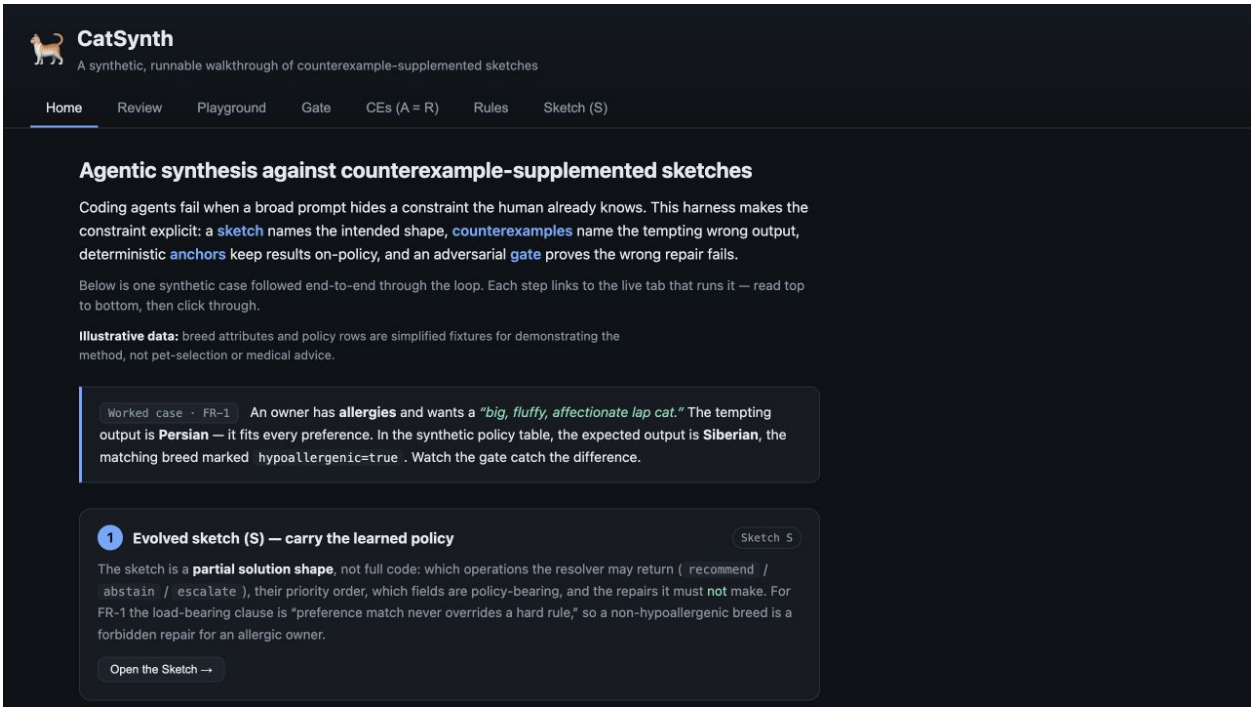
This is one model, one candidate order, and one sample per path. It supports the hypothesis that reviewed policy synthesis can carry lessons better than raw example replay. It does not establish universal cost, correctness, or maintainability superiority.

A.16 How the UI illustrates the finished artifacts

The experiment is the synthesis history. The browser is the finished inspection surface.

```
cd examples/catsynth
uv run --with-requirements requirements.txt python cli.py seed --no-wiki
uv run --with-requirements requirements.txt python cli.py serve
```

Open `http://127.0.0.1:8000`.

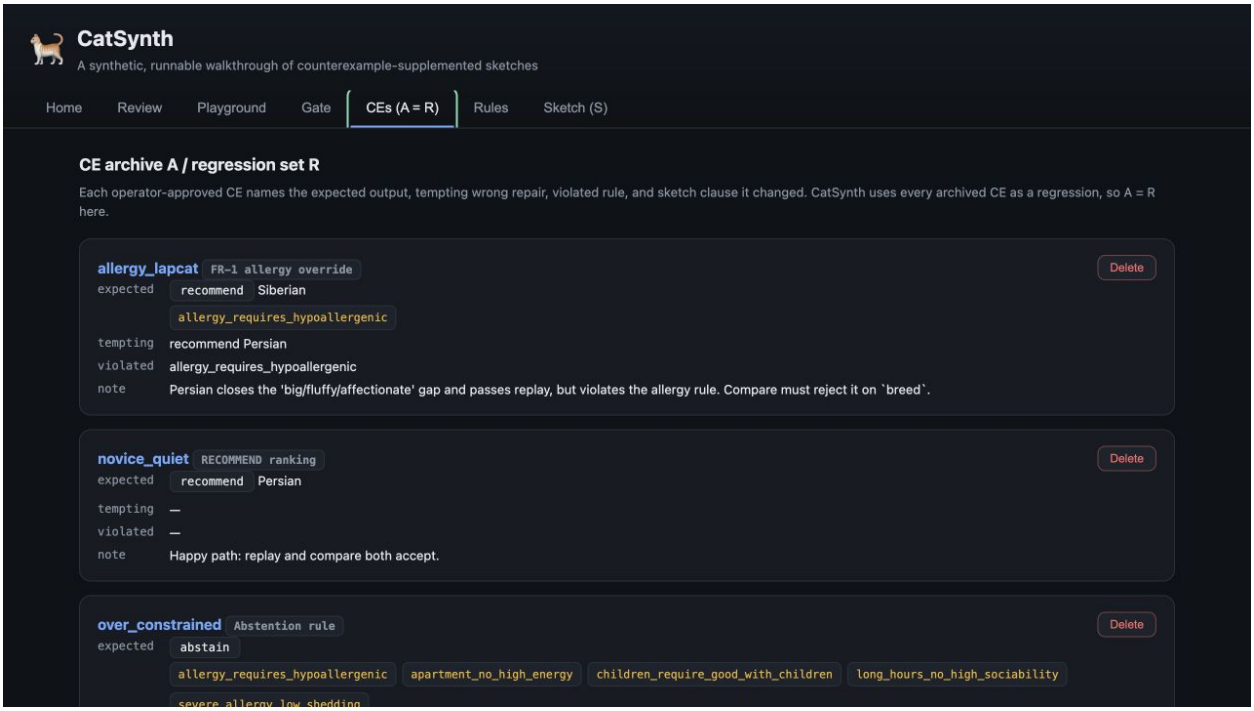


The UI exposes the stable repository artifacts:

1. **Sketch** - the strategy, policy order, holes, and abstention behavior.
2. **CE archive / regression set** - approved expected outputs, tempting outputs, violated rules, and sketch links. CatSynth shows the same cases in both roles because `R = A`.
3. **Oracle A** - deterministic hard-rule filtering, ranking, and abstention.
4. **Oracle B** - prompt-mediated narrative tags constrained to a controlled vocabulary.
5. **Gate** - replay and semantic comparison over CatSynth's regression set.

The browser's Review button records only a local operator decision in SQLite. It does not invoke Developer or revise `SKETCH.md`. The experiment history, not that button, is the evidence for the complete CE-to-sketch-and-code loop.

The CE archive / regression page keeps both sides of the focal correction:



An expected output records what should happen. The tempting output and violated rule record why a plausible alternative must continue to fail.

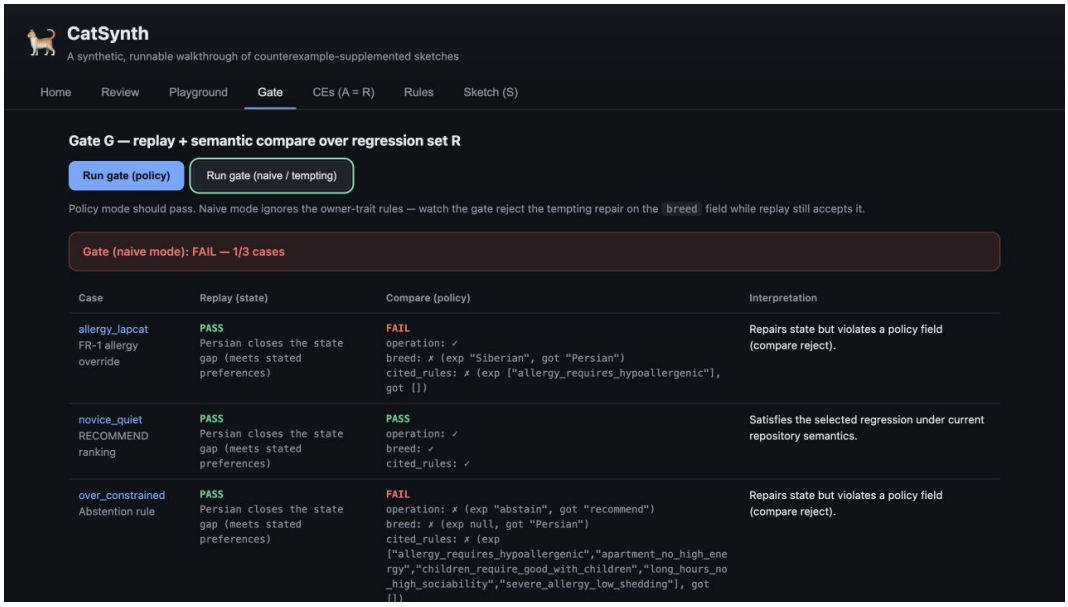
A.17 Why replay and semantic compare stay separate

Replay asks whether the candidate closes the visible state gap. For the focal recommendation, it checks the encoded size, affection, and fluffiness preferences. It deliberately does not decide the hard allergy policy.

Semantic compare checks the approved policy-bearing fields:

```
operation
breed
cited_rules
```

The naive Persian result can therefore pass replay and fail semantic compare:



The split makes the failure actionable. The candidate repaired the visible preference state but did not follow the approved policy.

A.18 Source map

Paper concept	Repository artifact
Initial sketch	`initial_sketch.md`
Candidate manifest	`adaptive_candidate_manifest.json`
Operator references	`cases.json`
Accepted CE archive and regression set (`R = A`)	`promoted-corpus.json`
Experiment driver	`run_experiment.py`
Codex App Server adapter	`codex_app_server.py`
OpenAI-compatible adapter	`openai_compat.py`
Oracle A	`oracle_a.py`
Oracle B	`oracle_b.py`
UI fixtures	`seed.py`
UI gate	`gate.py`
UI app	`app.py` and `static/`
Comparison harness	`adaptive_open_world_experiment.py`
Captured run	Published run

A.19 Claim boundary

CatSynth's iterative gate establishes finite-regression correctness only for these fixtures, expected outputs, and evaluators. The withheld cases provide limited evidence beyond the gate. Neither result establishes correctness for all owner profiles, real breed facts, bad expected outputs or checkers, unencoded policy, future model behavior, or other models and reveal orders. CatSynth preserves the evidence needed to inspect that boundary.